

**Mission: Not Autofix**

**Securing the World of Software with  
GitHub Advanced Security**

**Found by Autofix**



SECURITY  
HALL



## Mission Copilot Autofix

# Securing the World's Software with GitHub Advanced Security



**Randy Pagels**

Principal Trainer

Xebia USA | Microsoft Services





18th Annual  
**Orlando  
Code Camp** | 2026  
Sponsors, Organizers, and Partners





# Agenda

Welcome and Overview

The State of Security Today

Code Scanning, Code Quality & Autofix

Dependency Review

Secret Protection + Push Protection

Campaigns & Summary

Summary, Q&A, and Certification Exam Prep



# The State of Security Today

# Real-World Breaches

Even the biggest companies get caught off guard.

| Incident                                     | What Happened                                                                      | Impact                                                                                                    | How GHAS Could Have Helped                                                                                                                   |
|----------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SolarWinds (2020)</b>                     | Attackers injected malicious code into a build pipeline.                           | 18,000+ customers affected, including U.S. agencies. Cleanup costs estimated in the hundreds of millions. | <b>Code scanning + dependency review</b> could have flagged suspicious build-time behavior or risky third-party updates.                     |
| <b>Uber (2022)</b>                           | Hacker accessed systems via hardcoded credentials on GitHub.                       | Full infrastructure compromise, reputation damage, and regulatory scrutiny.                               | <b>Secret scanning + push protection</b> would have blocked or alerted on exposed secrets in code.                                           |
| <b>Toyota (2022)</b>                         | GitHub repo exposed private API keys tied to customer accounts.                    | 300,000+ users impacted, public trust eroded.                                                             | <b>Push protection</b> could have blocked the commit. Secret scanning would have surfaced the issue for remediation.                         |
| <b><u>Codecov</u> (2021)</b>                 | Bash uploader script was tampered with, infecting customers' CI pipelines.         | Data exposure for major customers. Long incident response effort.                                         | <b>Code scanning</b> might have detected script anomalies or unexpected behavior in dependency paths.                                        |
| <b>T-Mobile (2023)</b>                       | Massive data breach from compromised APIs and weak controls.                       | \$350M+ in breach costs. Legal action, fines, and brand damage.                                           | <b>Dependency scanning</b> , secret scanning, and code scanning could have closed weak points earlier in the lifecycle.                      |
| <b>Microsoft AI Data Exposure (Sep 2023)</b> | Overly-permissive Azure SAS token in a GitHub repo exposed 38 TB of internal data. | Exposed credentials, internal Teams messages, and private keys. Regulatory scrutiny followed.             | <b>Secret scanning + push protection</b> could have flagged the token before commit. Code scanning might have caught abnormal data patterns. |

# Real-World Breaches

Even the biggest companies get caught off guard.

| Incident                                     | What Happened                                                                      | Impact                                                                                                    | How GHAS Could Have Helped                                                                                                                   |
|----------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SolarWinds (2020)</b>                     | Attackers injected malicious code into a build pipeline.                           | 18,000+ customers affected, including U.S. agencies. Cleanup costs estimated in the hundreds of millions. | <b>Code scanning + dependency review</b> could have flagged suspicious build-time behavior or risky third-party updates.                     |
| <b>Uber (2022)</b>                           | Hacker accessed systems via hardcoded credentials on GitHub.                       | Full infrastructure compromise, reputation damage, and regulatory scrutiny.                               | <b>Secret scanning + push protection</b> would have blocked or alerted on exposed secrets in code.                                           |
| <b>Toyota (2022)</b>                         | GitHub repo exposed private API keys tied to customer accounts.                    | 300,000+ users impacted, public trust eroded.                                                             | <b>Push protection</b> could have blocked the commit. Secret scanning would have surfaced the issue for remediation.                         |
| <b><u>Codecov</u> (2021)</b>                 | Bash uploader script was tampered with, infecting customers' CI pipelines.         | Data exposure for major customers. Long incident response effort.                                         | <b>Code scanning</b> might have detected script anomalies or unexpected behavior in dependency paths.                                        |
| <b>T-Mobile (2023)</b>                       | Massive data breach from compromised APIs and weak controls.                       | \$350M+ in breach costs. Legal action, fines, and brand damage.                                           | <b>Dependency scanning</b> , secret scanning, and code scanning could have closed weak points earlier in the lifecycle.                      |
| <b>Microsoft AI Data Exposure (Sep 2023)</b> | Overly-permissive Azure SAS token in a GitHub repo exposed 38 TB of internal data. | Exposed credentials, internal Teams messages, and private keys. Regulatory scrutiny followed.             | <b>Secret scanning + push protection</b> could have flagged the token before commit. Code scanning might have caught abnormal data patterns. |

# Real-World Breaches

Even the biggest companies get caught off guard.

| Incident                              | What Happened                                                                      | Impact                                                                                                    | How GHAS Could Have Helped                                                                                                                   |
|---------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| SolarWinds (2020)                     | Attackers injected malicious code into a build pipeline.                           | 18,000+ customers affected, including U.S. agencies. Cleanup costs estimated in the hundreds of millions. | <b>Code scanning + dependency review</b> could have flagged suspicious build-time behavior or risky third-party updates.                     |
| Uber (2022)                           | Hacker accessed systems via hardcoded credentials on GitHub.                       | Full infrastructure compromise, reputation damage, and regulatory scrutiny.                               | <b>Secret scanning + push protection</b> would have blocked or alerted on exposed secrets in code.                                           |
| <b>Toyota (2022)</b>                  | GitHub repo exposed private API keys tied to customer accounts.                    | 300,000+ users impacted, public trust eroded.                                                             | <b>Push protection</b> could have blocked the commit. Secret scanning would have surfaced the issue for remediation.                         |
| <u>Codecov</u> (2021)                 | Bash uploader script was tampered with, infecting customers' CI pipelines.         | Data exposure for major customers. Long incident response effort.                                         | <b>Code scanning</b> might have detected script anomalies or unexpected behavior in dependency paths.                                        |
| T-Mobile (2023)                       | Massive data breach from compromised APIs and weak controls.                       | \$350M+ in breach costs. Legal action, fines, and brand damage.                                           | <b>Dependency scanning</b> , secret scanning, and code scanning could have closed weak points earlier in the lifecycle.                      |
| Microsoft AI Data Exposure (Sep 2023) | Overly-permissive Azure SAS token in a GitHub repo exposed 38 TB of internal data. | Exposed credentials, internal Teams messages, and private keys. Regulatory scrutiny followed.             | <b>Secret scanning + push protection</b> could have flagged the token before commit. Code scanning might have caught abnormal data patterns. |



# Real-World Breaches

Even the biggest companies get caught off guard.

| Incident                                     | What Happened                                                                      | Impact                                                                                                    | How GHAS Could Have Helped                                                                                                                   |
|----------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| SolarWinds (2020)                            | Attackers injected malicious code into a build pipeline.                           | 18,000+ customers affected, including U.S. agencies. Cleanup costs estimated in the hundreds of millions. | <b>Code scanning + dependency review</b> could have flagged suspicious build-time behavior or risky third-party updates.                     |
| Uber (2022)                                  | Hacker accessed systems via hardcoded credentials on GitHub.                       | Full infrastructure compromise, reputation damage, and regulatory scrutiny.                               | <b>Secret scanning + push protection</b> would have blocked or alerted on exposed secrets in code.                                           |
| Toyota (2022)                                | GitHub repo exposed private API keys tied to customer accounts.                    | 300,000+ users impacted, public trust eroded.                                                             | <b>Push protection</b> could have blocked the commit. Secret scanning would have surfaced the issue for remediation.                         |
| <u>Codecov</u> (2021)                        | Bash uploader script was tampered with, infecting customers' CI pipelines.         | Data exposure for major customers. Long incident response effort.                                         | <b>Code scanning</b> might have detected script anomalies or unexpected behavior in dependency paths.                                        |
| T-Mobile (2023)                              | Massive data breach from compromised APIs and weak controls.                       | \$350M+ in breach costs. Legal action, fines, and brand damage.                                           | <b>Dependency scanning</b> , secret scanning, and code scanning could have closed weak points earlier in the lifecycle.                      |
| <b>Microsoft AI Data Exposure (Sep 2023)</b> | Overly-permissive Azure SAS token in a GitHub repo exposed 38 TB of internal data. | Exposed credentials, internal Teams messages, and private keys. Regulatory scrutiny followed.             | <b>Secret scanning + push protection</b> could have flagged the token before commit. Code scanning might have caught abnormal data patterns. |

# Why GitHub Advanced Security?

**Traditional AppSec = too little, too late**

Security becomes a backlog item

**80–90% of app code is open source**

Unmonitored dependencies = major risk

**Developer mistakes (exposed tokens, secrets, unsafe code)**

Cause real breaches

**570x more developers than security researchers**

We need scale

**GHAS + GitHub Copilot = scalable**

Proactive protection built into your workflow

A new vision for AppSec





# Found means fixed

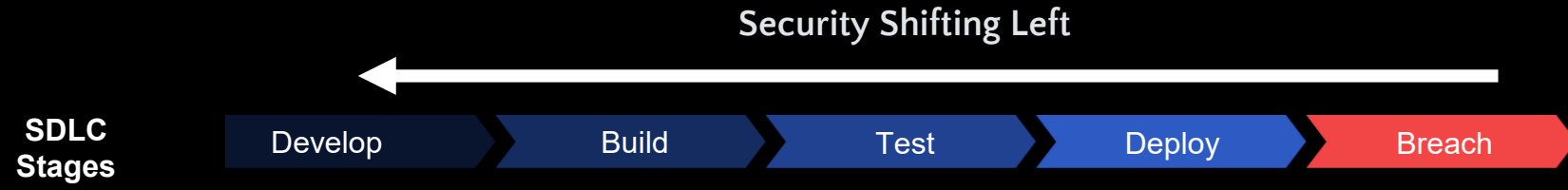


***MISSION: X IMPOSSIBLE***

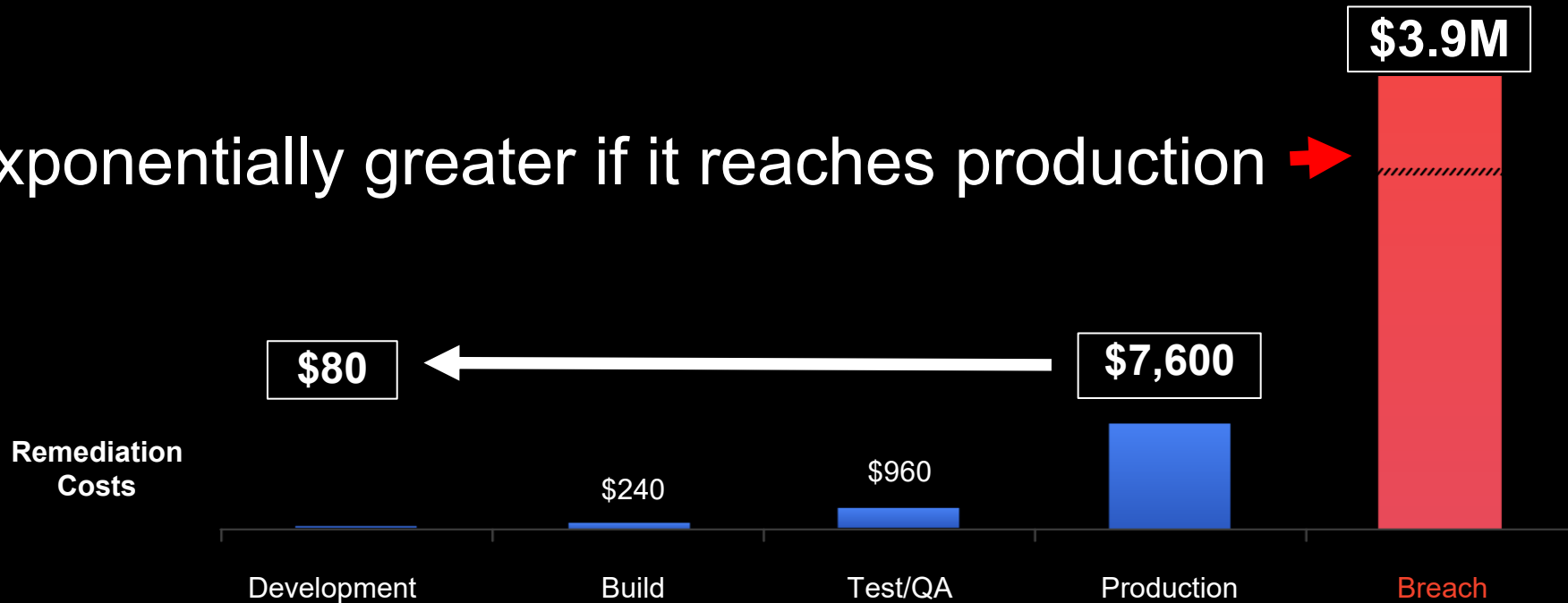
**Detection without remediation  
is just another to-do list you  
don't have time for...**



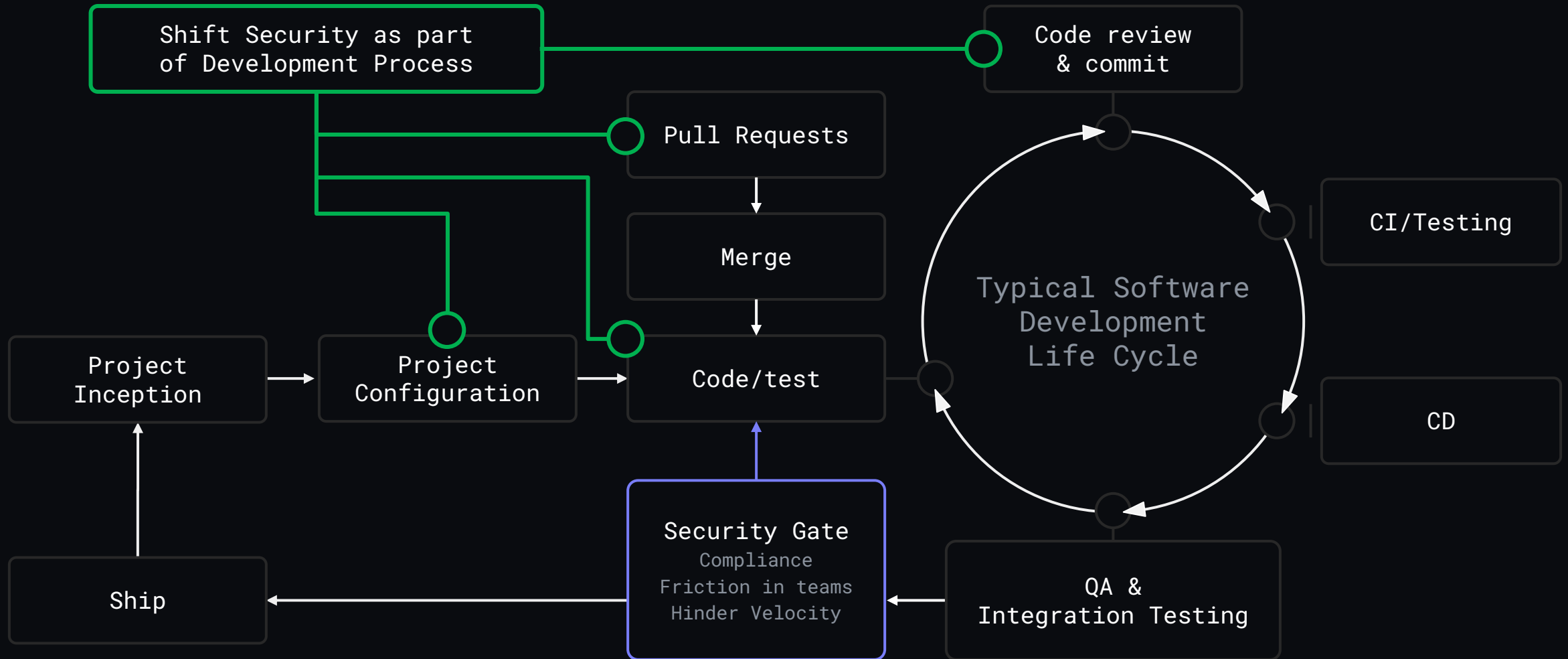
# Everyone wants to shift security left...



Damage is exponentially greater if it reaches production →

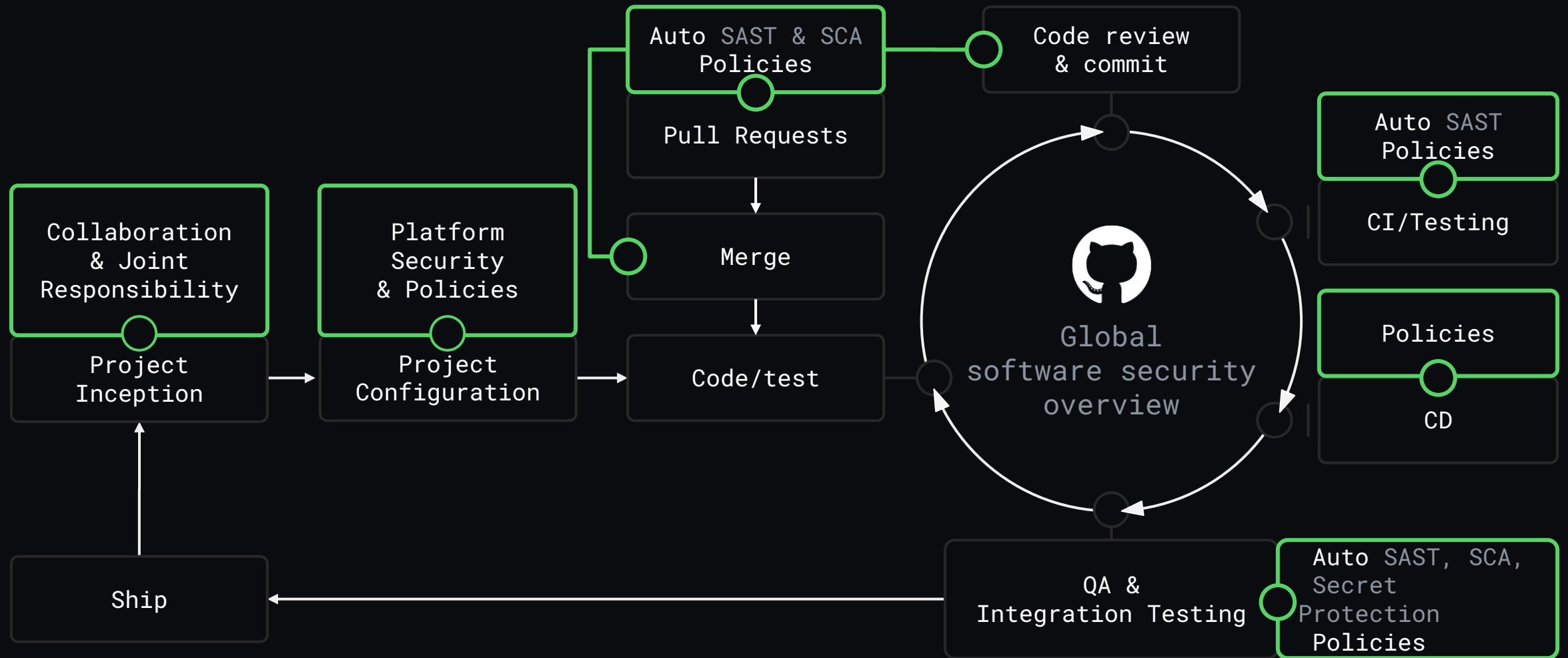


# Application Security in the SDLCs





# Application Security using DevSecOps



# Three Aspects of Security Protection

## Detection

- ✓ Find security issues
- ✓ Alert team
- ✓ Highlight problems
  - ✓ exposed secrets
  - ✓ known vulnerabilities
  - ✓ risky code patterns

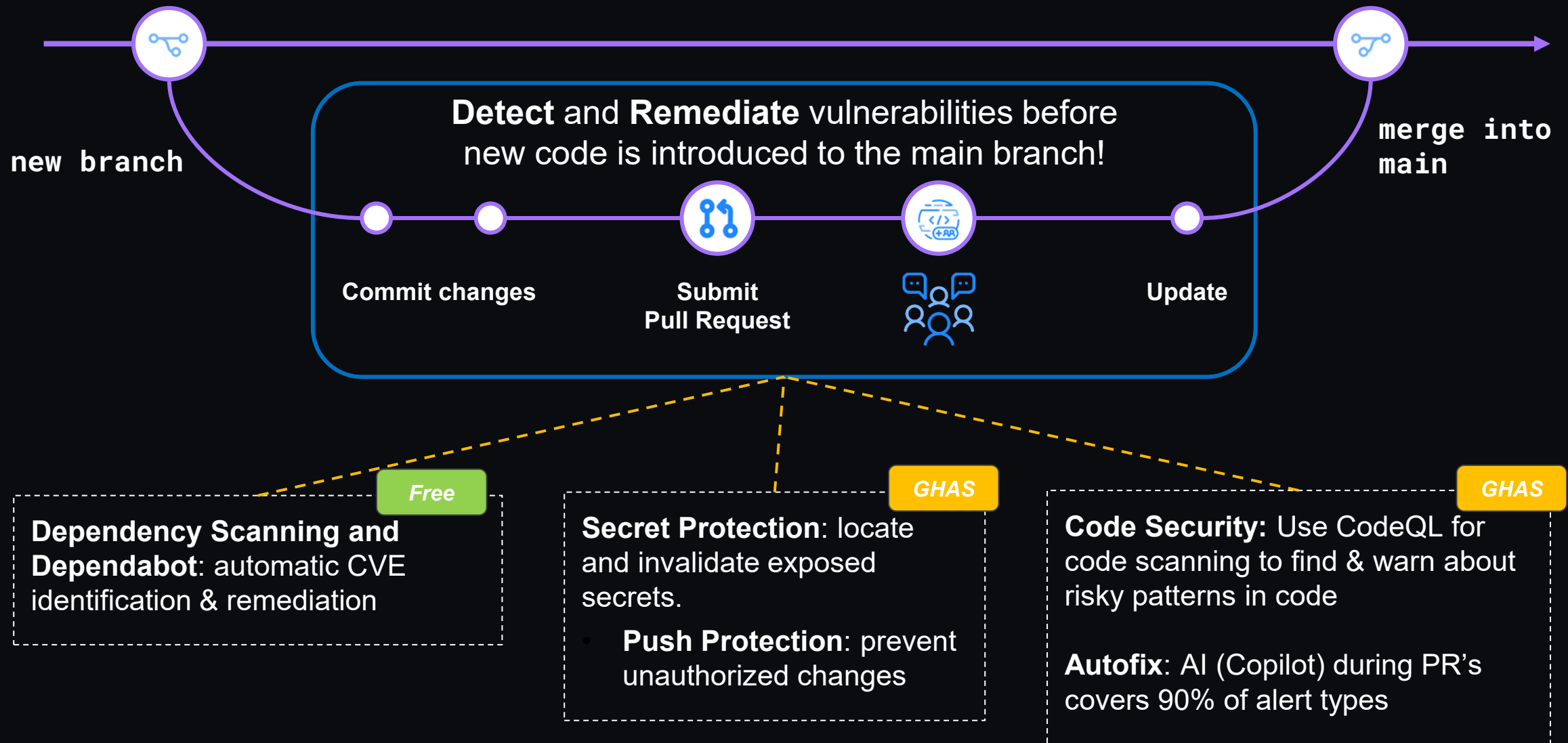
## Remediation

- ✓ Fix security issues quickly
- ✓ Suggest and Automate fixes

## Prevention

- ✓ Blocks new security issues
- ✓ Use automated checks and rules to keep bad code and secrets out

# Developer First Security – Shift Left



# GitHub Advanced Security



## Code scanning

Analyzes code for vulnerabilities using **CodeQL** and provides alerts in pull requests or on push



## Code quality

Automatically identifies and remediates maintainability issues like complex methods and duplicated code



## Dependency scanning

Automatically detects known vulnerabilities in existing dependencies using Dependabot alerts.



## Secret Protection

Detects hardcoded secrets in your codebase, both in commit history and before they're pushed.



# Demo

- ✓ **Setup and Overview**
- ✓ **Reviewing Alerts**



**GitHub  
Advanced  
Security**



**Code Security**

# Code Scanning

github-advanced-security bot

The vulnerability in the code is due to the fact that user-provided input is directly used in HTTP response without any sanitization. This can lead to a cross-site scripting (XSS) attack if the user input contains malicious scripts.

To fix this, we need to sanitize the input before using it in the HTTP response. One way to do this is to use the `escape-html` library, which can escape any special characters.

Copilot Autofix AI

```
app.get(  , (req, res) => {  
15 -   res.send('Hello ${req.query.name}!');  
15 +   res.send('Hello ${escape(req.query.name)}!');  
16 }
```

Copilot Autofix

## Apply fixes in seconds

Spend less time fixing vulnerabilities  
and more time building features

# Code Scanning

- Find vulnerabilities before they are merged into the code base with automated CodeQL scans
- Integrate results directly into the developer workflow
- Run custom queries and the community-powered GitHub query set
- Extensible, with support for other 3<sup>rd</sup> party SAST tools

Code scanning alerts / #14

## Database query built from user-controlled sources

 Open in `main` 14 minutes ago

 Speed up the remediation of this alert with [Copilot Autofix for CodeQL](#)  Generate fix

routes/search.ts:23

```
20 return (req: Request, res: Response, next: NextFunction) => {
21   let criteria: any = req.query.q === 'undefined' ? '' : req.query.q ?? ''
22   criteria = (criteria.length <= 200) ? criteria : criteria.substring(0, 200)
23   models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE`
```

This query string depends on a user-provided value.

CodeQL [Show paths](#)

```
24 .then(([products]: any) => {
25   const dataString = JSON.stringify(products)
26   if (challengeUtils.notSolved(challenges.unionSqlInjectionChallenge)) { // vuln-code-snippet hide-
```

| Tool   | Rule ID          | Query                       |
|--------|------------------|-----------------------------|
| CodeQL | js/sql-injection | <a href="#">View source</a> |

If a database query (such as a SQL or NoSQL query) is built from user-provided data without sufficient sanitization, a malicious user may be able to run malicious database queries.

Severity: **High**

Affected branches: **main**

Developer: [Link a branch](#) [create a new branch](#)

Security can: [No campaign](#)

Tags: [security](#)

# Code Scanning with Copilot Autofix

Code scanning alerts / #15

Database query built from user-controlled sources

Dismiss alert

Open in main 17 minutes ago

Copilot Autofix AI is generating a fix suggestion...

routes/updateProductReviews.ts:18

```
15 return (req: Request, res: Response, next: NextFunction) => {
16   const user = security.authenticatedUsers.from(req) // vuln-code-snippet vuln-line forgedReviewChallen
17   db.reviewsCollection.update( // vuln-code-snippet neutral-line forgedReviewChallenge
18     { _id: req.body.id }, // vuln-code-snippet vuln-line noSqlReviewsChallenge forgedReviewChallenge

This query object depends on a user-provided value.
This query object depends on a user-provided value.

CodeQL Show paths

19   { $set: { message: req.body.message } },
20   { multi: true } // vuln-code-snippet vuln-line noSqlReviewsChallenge
21   ).then(
```

| Tool   | Rule ID          | Query                       |
|--------|------------------|-----------------------------|
| CodeQL | js/sql-injection | <a href="#">View source</a> |

If a database query (such as a SQL or NoSQL query) is built from user-provided data without sufficient sanitization, a malicious user may be able to run malicious database queries.

Severity

High

Affected branches

main default

First detected 10 minutes ago

Development

Link a branch, pull request, or [create a new branch](#) to start working on this alert.

Security campaigns

No campaign is currently tracking this alert.

Tags

security



# What is code scanning with CodeQL?



# CodeQL

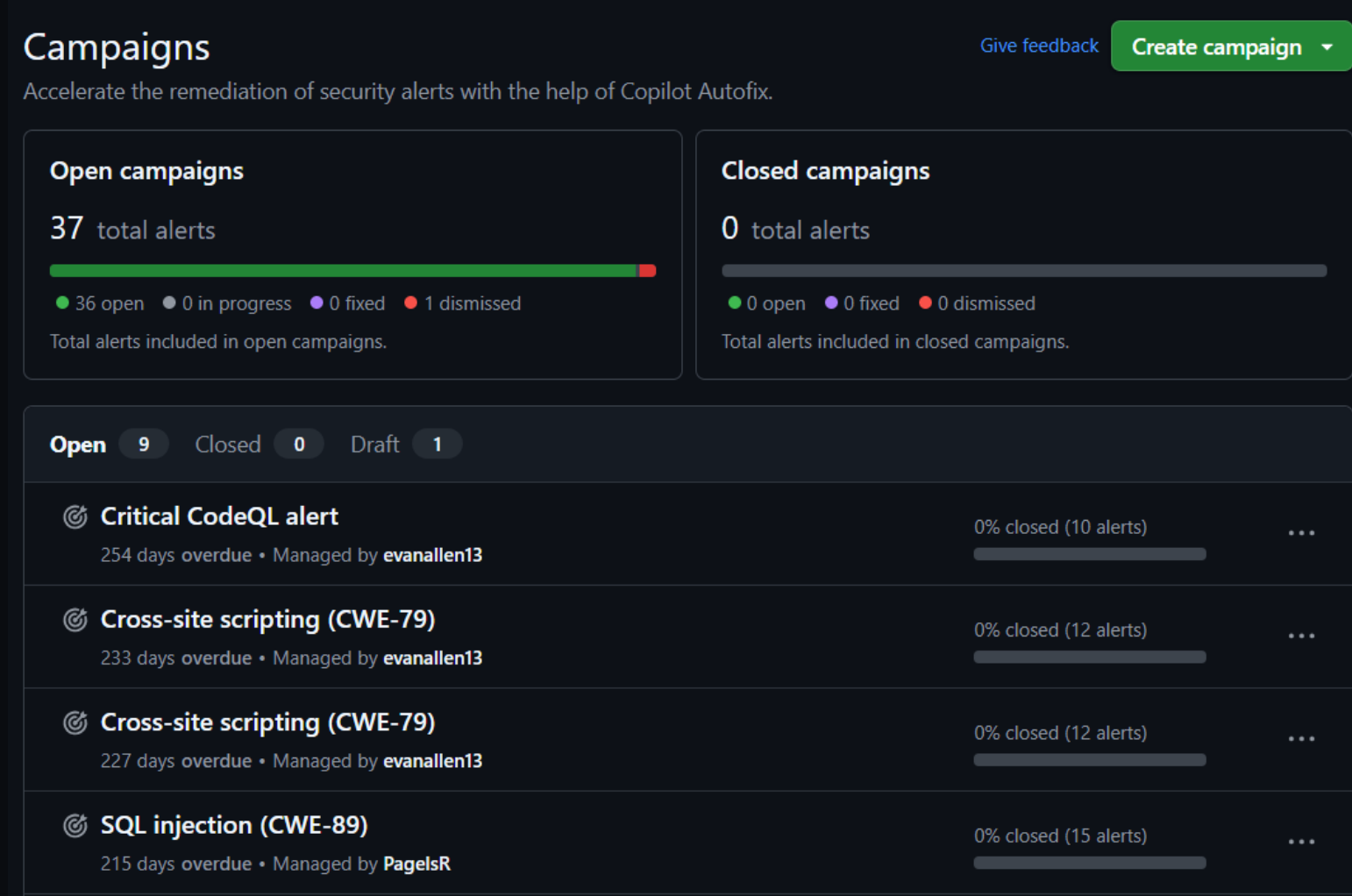
- Queries are open source
- Active community
- CodeQL supports both compiled and interpreted languages
- Supported Languages
  - C/C++
  - C#
  - Go
  - Java/Kotlin
  - JavaScript/TypeScript
  - Python
  - Ruby
  - Rust (public preview)
  - Swift
  - GitHub Actions workflows

# Security Campaigns for Copilot Autofix



# Security Campaigns

**Rapidly reduce  
your security  
debt**



# Security Campaigns with Copilot Autofix

**Rapidly reduce  
your security  
debt**

## SQL injection (CWE-89)-12.23.2025

[Give feedback](#)

Managed by **PagelsR**

Remediating SQL injection vulnerabilities prevents unauthorized database access, data theft, and manipulation, ensuring regulatory compliance, maintaining application integrity, and enhancing user trust.

### Campaign progress

[View organization-level campaign](#)

0% (0 alerts)

6 alerts left

Campaign started 223 days ago

### Status

150 days left

Due date is Wed, Dec 31

### Copilot Autofix

6 supported alerts

Copilot Autofix will try to suggest fixes for the supported alerts. Read more about [Copilot Autofix](#).

☒ 3 of 6 selected

Commit autofixes

Close alerts

☒ ☐ Database query built from user-controlled sources **High** Autofix

#25 · Opened 3 years ago · Detected by CodeQL in routes/updateProductReviews.ts:18

☒ ☐ Database query built from user-controlled sources **High** Autofix

#24 · Opened 3 years ago · Detected by CodeQL in routes/search.ts:22

☒ ☐ Database query built from user-controlled sources **High** Autofix

#23 · Opened 3 years ago · Detected by CodeQL in routes/login.ts:35

☐ ☐ Database query built from user-controlled sources **High** Autofix

#21 · Opened 3 years ago · Detected by CodeQL in routes/likeProductReviews.ts:31

Commit to new branch

Commit to existing branch



**GitHub  
Advanced  
Security**



**Code quality**

# Code Quality

- Identifies and remediates maintainability issues like complex methods and duplicated code.
- Uses Copilot Autofix to generate production-ready refactors in seconds.
- Reduces technical debt by catching architectural "smells" before they reach production.
- Streamlines code reviews by automating the detection of non-compliant patterns.

## Property access on null or undefined

Last scan: 4 days ago

Trying to access a property of "null" or "undefined" will result in a runtime exception.

Show more ▾

Open 1 Dismissed 0

test/cypress/e2e/profile.spec.ts:79 Autofix

Open pull request

```
74      cy.visit('http://htmledit.squarefree.com')
75      /* The script executed below is equivalent
       to pasting this string into
       http://htmledit.squarefree.com: */
76      /* <form
       action="http://localhost:3000/profile"
       method="POST"><input type="hidden" name="username"
       value="CSRF"/><input type="submit"/></form>
       <script>document.forms[0].submit();</script> */
77      - let document: any
78      - cy.window().then(() => {
79      -     document
80      -     .getElementsByName('editbox')[0]
81      -     .contentDocument.getElementsByName(
82      -     'ta'
```

```
74      cy.visit('http://htmledit.squarefree.com')
75      /* The script executed below is equivalent
       to pasting this string into
       http://htmledit.squarefree.com: */
76      /* <form
       action="http://localhost:3000/profile"
       method="POST"><input type="hidden" name="username"
       value="CSRF"/><input type="submit"/></form>
       <script>document.forms[0].submit();</script> */
77      + cy.window().then((win) => {
78      +     win.document
79      +     .getElementsByName('editbox')[0]
80      +     .contentDocument.getElementsByName(
81      +     'ta'
```

# Demo

## ✓ Code Security & Quality

- ✓ (Alerts)
- ✓ (Copilot Autofix)
- ✓ (Campaigns)



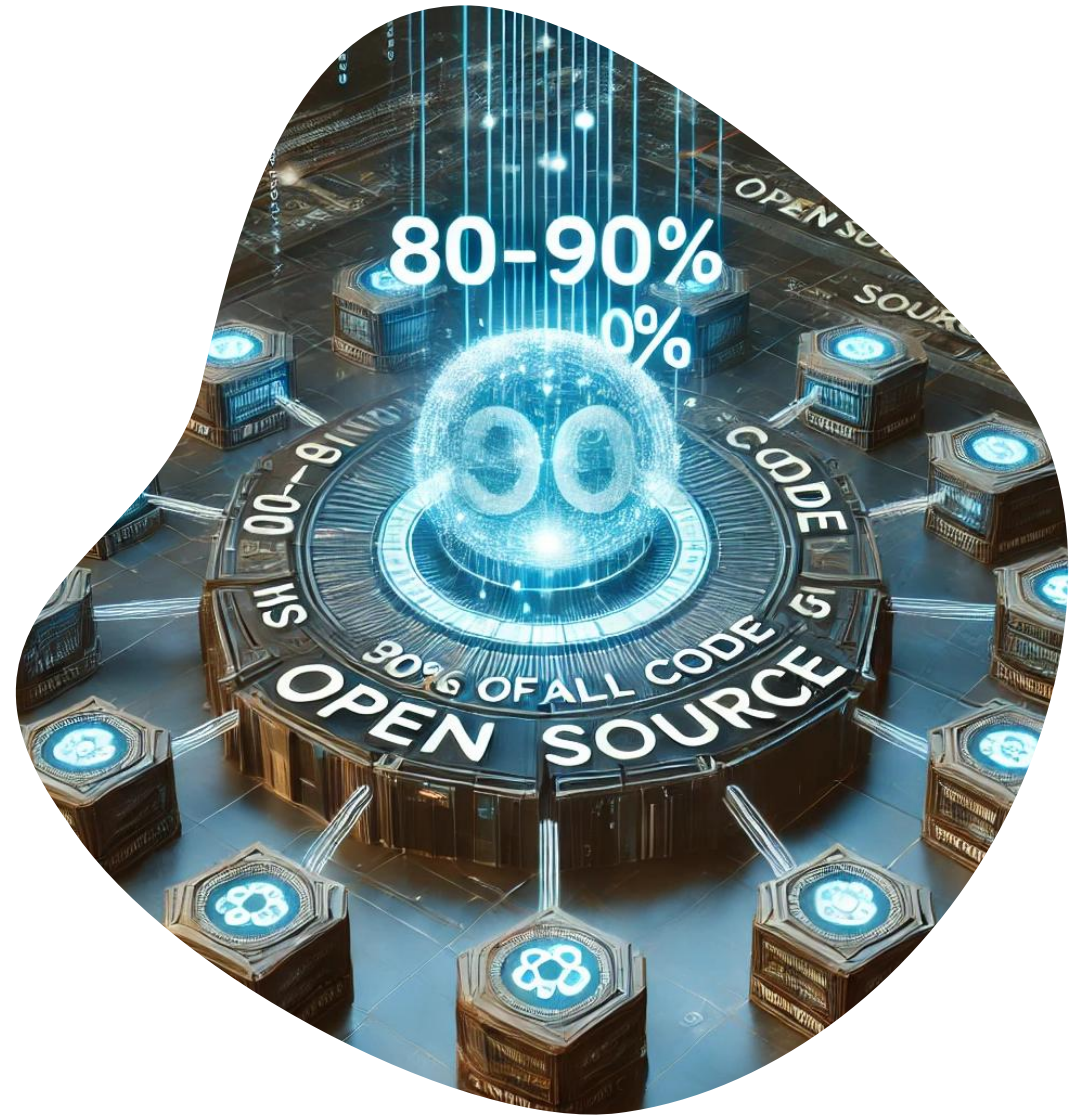
**GitHub  
Advanced  
Security**



**Dependency  
scanning**



**80-90% of all  
code in distributed  
applications is  
Open Source**

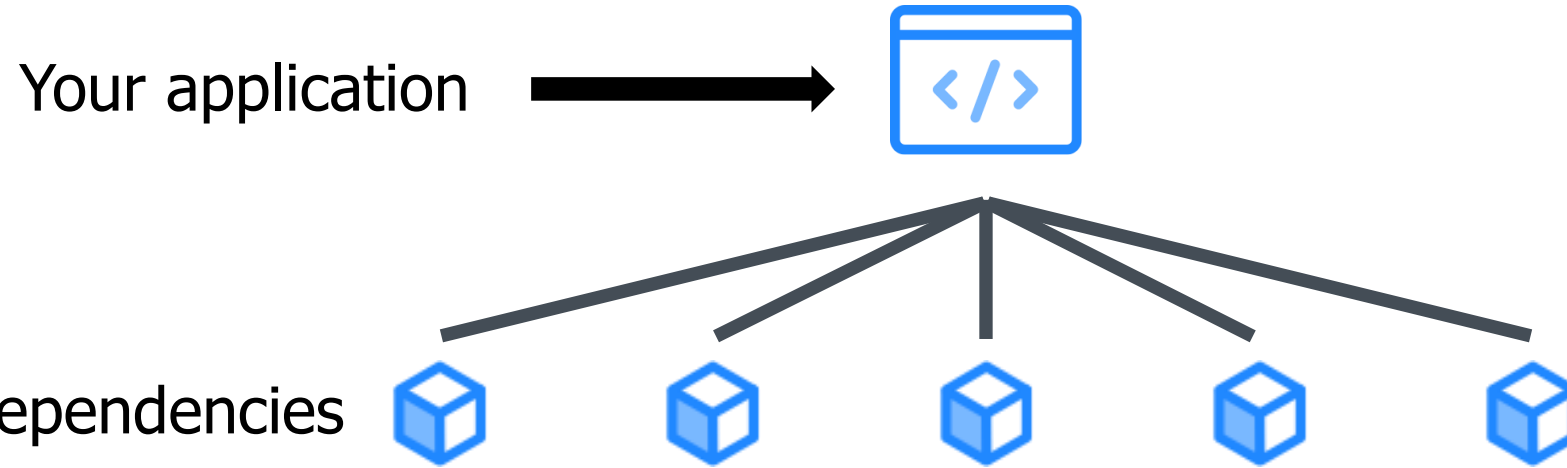




# Dependency graph

- Summary of manifest and lock files showing both:
  - Ecosystems and packages the repository depends on
  - Repositories and packages that are dependents
- Central to supply chain security
- Direct and transitive dependencies
- Growing language and package support

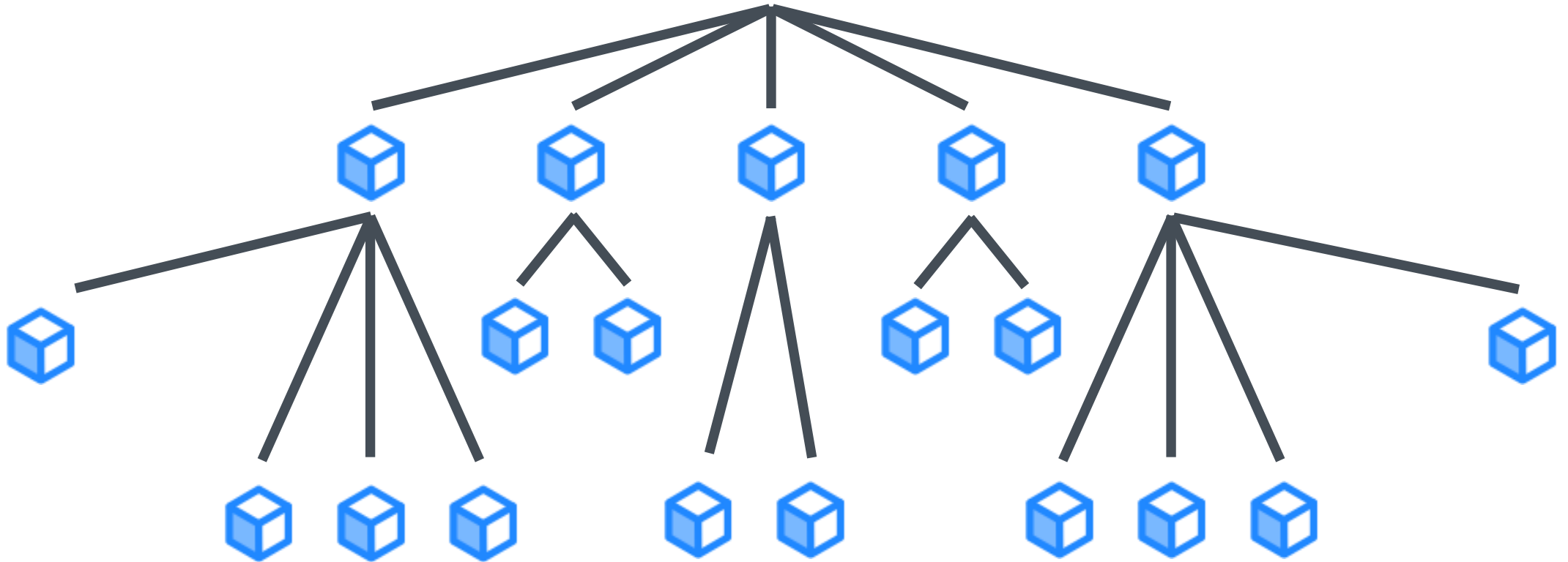
# Securing your Supply Chain



Dependency Chain starts with Top-Level Dependencies

# Securing your Supply Chain

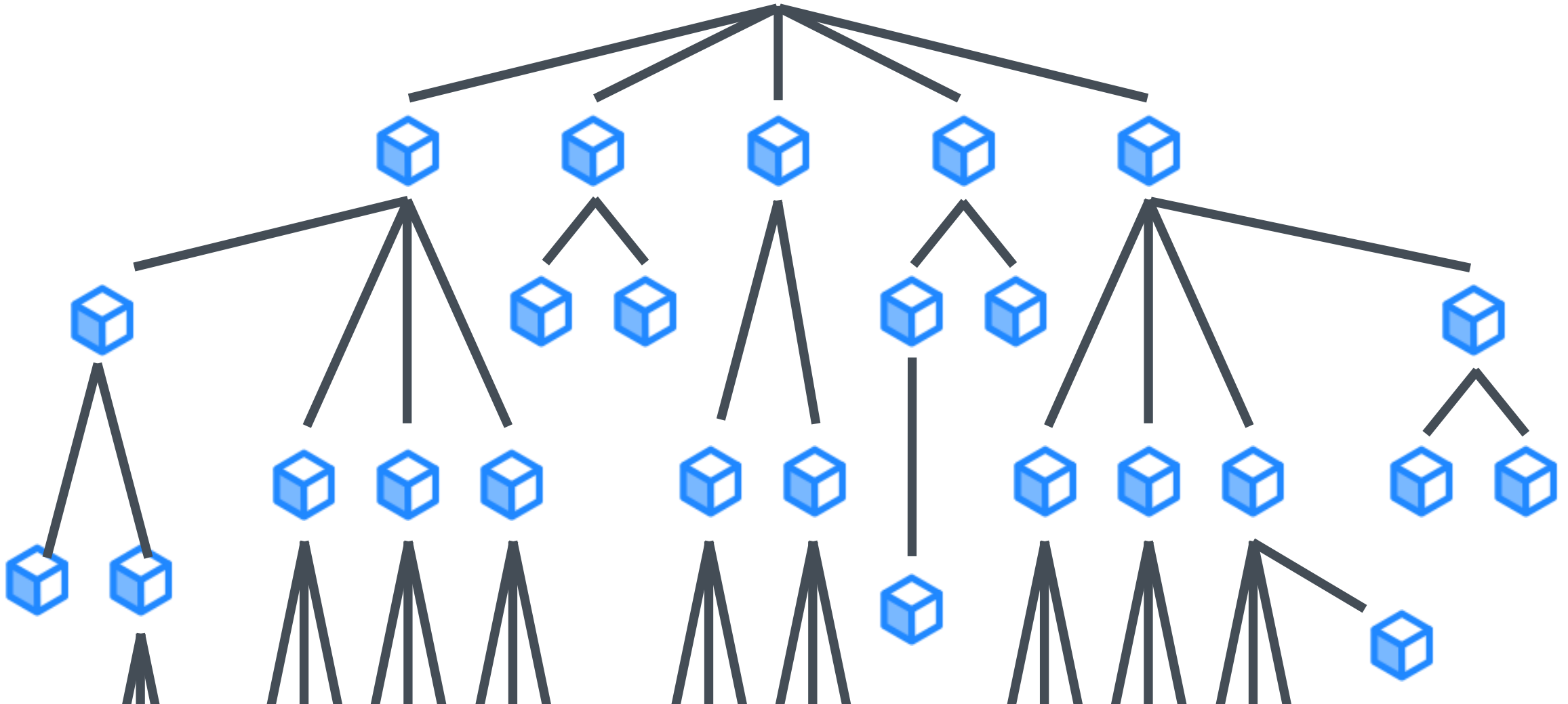
Your application



indirect (transitive) dependencies

# Securing your Supply Chain

Your application

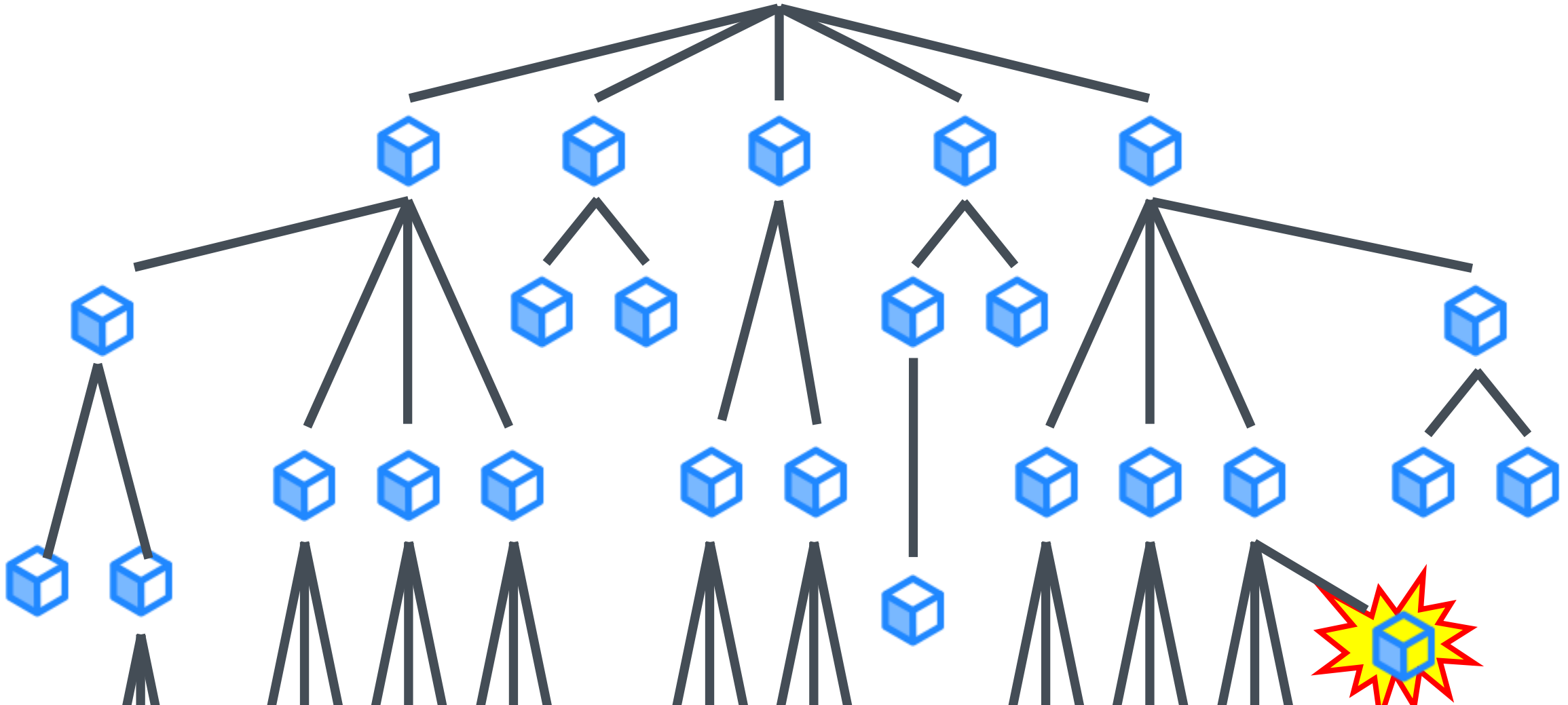


# Securing your Supply Chain

Your application



Who's responsible for security?  
Who's tracking packages?  
Who's keeping components updated?





# Dependabot

Automatically update vulnerable and out-of-date dependencies



## Automated pull requests for security & version updates

Dependabot scans for security issues or outdated dependencies and automatically opens pull requests with recommended fixes.



## Integrated with developer workflow

Dependabot works right in your existing workflow, making updates and fixes fast and simple..



## Rich vulnerability data

GitHub pulls vulnerability details from multiple trusted sources and lists them in the Advisory Database.

Dependabot alerts / #1

## Verification Bypass in jsonwebtoken #1

 **Open** Opened 3 hours ago on jsonwebtoken (npm) · package.json



**Bump jsonwebtoken from 0.4.0 to 9.0.0**

Merging this pull request would fix 4 Dependabot alerts on jsonwebtoken in package.json.



**Review security update**

Package

 jsonwebtoken (npm)

Affected versions

< 4.2.2

Patched version

4.2.2 

Versions 4.2.1 and earlier of jsonwebtoken are affected by a verification bypass vulnerability. This is a result of weak validation of the JWT algorithm type, occurring when an attacker is allowed to arbitrarily specify the JWT algorithm.

### Recommendation

Update to version 4.2.2 or later.



dependabot (bot) opened this 3 hours ago

Severity

**Critical**

EPSS score

41.149% (97th)

Tags

Runtime de

Direct

Weaknesses

► CWE-20

Related alert

! jsonwebtoken

! jsonwebtoken

! jsonwebtoken



# Demo

## Dependency Review

- ✓ (Alert)
- ✓ (Blocking)



# **GitHub Advanced Security**



## **Secret Protection**

# Secret Protection

Find and manage hard-coded secrets



**Identifies secrets as early as possible**  
Alerts you as soon as secrets appear in your git history.



**Community of Secret Protection partners**  
Detects partner secret formats in every commit.



**Define custom patterns**  
Lets you scan for your own secret patterns.



**Supports public and private repos**  
Protects both public and private repositories.

Secret scanning alerts / #5

## GitHub Personal Access Token

[Open](#) GitHub detected a secret 4 hours ago

### Validity unknown

This secret is not currently active on github.com, but it may have been active in the past, Last verified 2 hours ago or may be an active secret from another instance.

ghp\_hnsKbni4wU88r3wpdY51Ln0IQZR8fS4SgJWU

### Remediation steps

Follow the steps below before you close this alert.

- 1 Rotate the secret if it's in use to prevent breaking workflows.
- 2 Revoke this GitHub Personal Access Token through GitHub to prevent unauthorized access. [Learn more about GitHub tokens.](#)
- 3 Check security logs for potential breaches.
- 4 Close the alert as revoked.

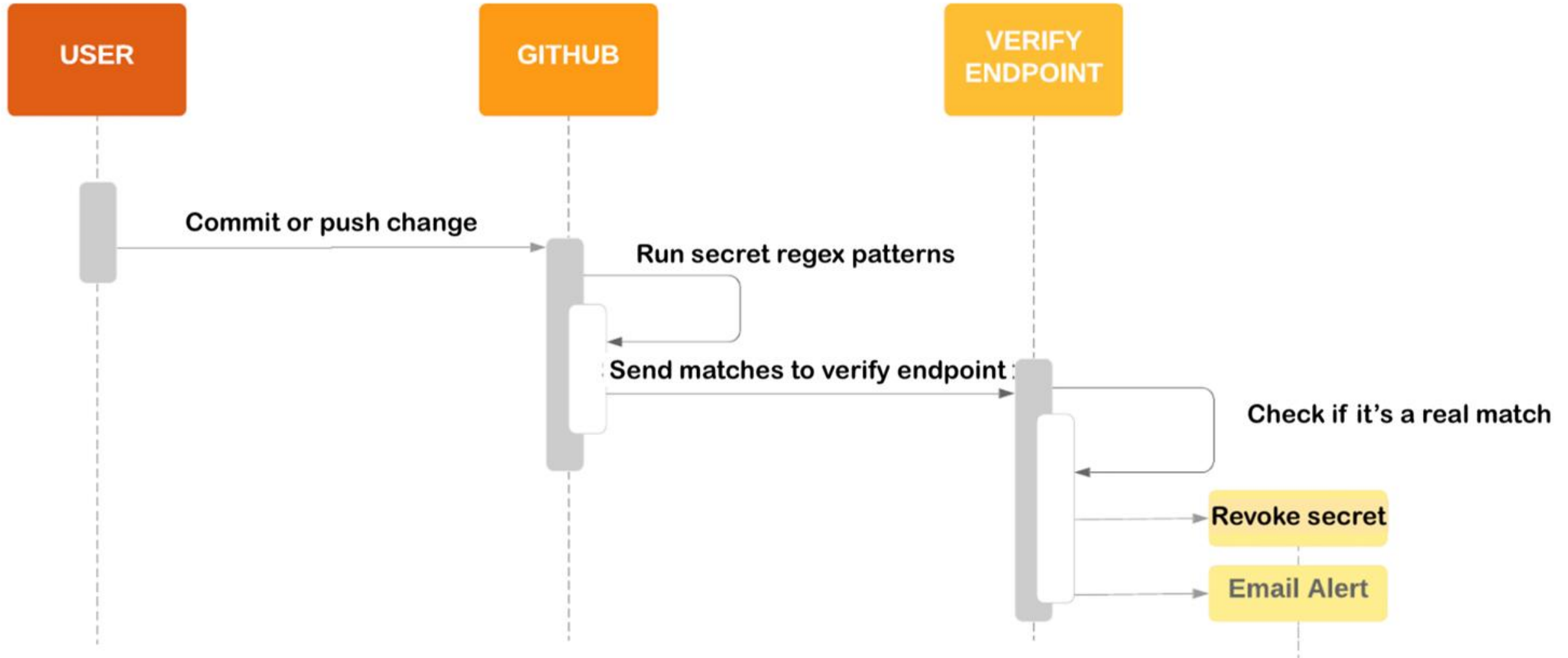
Detected in 1 location

# Secret Protection Partners (120+)



# Secret Scanning Lifecycle

**Detect → Block → Fix → Revoke → Resolve → Track**





# Validity Checks

- Some detected secrets are validated to reduce false positives.
- This increases developer trust in alerts.
- Not all secret types support validation.

## Google API Key

[Open](#) GitHub detected a secret yesterday

Publicly leaked inactive secret

Last verified 2 days ago

AIzaSyDvc2t8M5wjfDonZ1e4xG3wjFcCWHIN0yE

## Password

[Open](#) GitHub detected a secret yesterday

Validity unknown

iBurri3dMySe1fInTheB4ckyard!

 Detected by [Copilot Secret Scanning](#)

# Custom Patterns

- Custom patterns can be defined for your enterprise, organization, or repository
- These patterns are specified using regex patterns and uses the Hyperscan library

## Security & analysis / New custom pattern

### Pattern name

This cannot be edited after saving.

### Secret format

The pattern for the secret, specified as a regular expression. [Learn more.](#)

> More options

### Test string (required) - 1 match

Provide a sample test string to make sure your configuration matches the patterns you expect.

Save and dry run

Save and dry run

Provide a sample test string to make sure your configuration matches the patterns you expect.

# Push Protection

- Prevent secrets from being pushed into organization or repository
- Can be enabled at the organization level or the repository level
- Push protection blocks new secrets at push time
- Existing secrets are managed through alerts and remediation, not repeated push blocks

! [Secret scanning](#) found a **GitHub Personal Access Token** secret on [line 2](#).

Allowing this secret risks exposure. Instead, consider [removing the secret from your commit and commit history](#).

Exposing this secret can allow someone to:

- Verify the identity of this **GitHub Personal Access Token** secret
- Know which resources this secret can access
- Act on behalf of the secret's owner
- Push this secret to this repository without being blocked

- ☒ It's used in tests  
The secret poses no risk. If anyone finds it, they cannot do any damage or gain access to sensitive information.
- ☐ It's a false positive  
The detected string is not a secret.
- ☐ I'll fix it later  
The secret is real, I **understand the risk**, and I will need to revoke it. This will open a security alert and notify admins of this repository.

Cancel

Allow Secret

# Demos

## ✓ Secret Protection

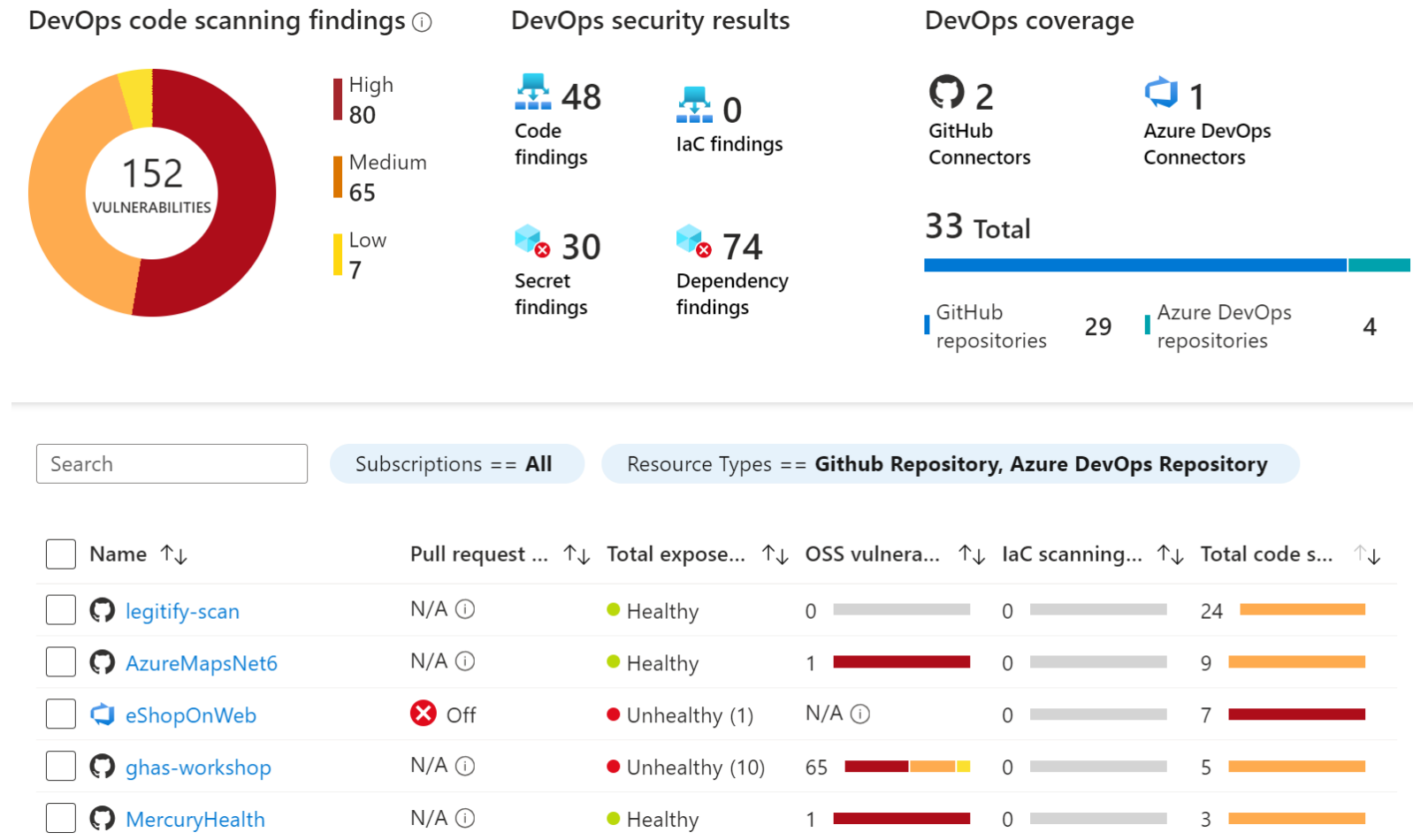
- ✓ (Push Protection)
- ✓ (Default)
- ✓ (Generic)



# Microsoft Defender for Cloud

**DevOps Security** – Insights across multi-pipeline environments to prioritize remediation and apply security guardrails

- Strengthen configuration of cloud resources throughout SDLC
- Provide unified, end-to-end visibility across departments
- Prioritize remediation of critical code issues using actionable feedback



<https://www.microsoft.com/security/business/cloud-security/microsoft-defender-devops> &  
<https://mslearn.cloudguides.com/guides/Unify DevOps security management with Microsoft Defender for Cloud>

# Resources

**GitHub Advanced Security:** <https://github.com/advanced-security>

**GitHub Advisory Database:** <https://github.com/advisories>

## About Dependabot alerts

- <https://docs.github.com/en/code-security/dependabot/dependabot-alerts/about-dependabot-alerts>

## About Secret Protection

- <https://docs.github.com/en/code-security/secret-scanning/about-secret-scanning>

## About code scanning

- <https://docs.github.com/en/code-security/code-scanning/introduction-to-code-scanning/about-code-scanning>

## Learning Path - Secure at every step

- <https://resources.github.com/learn/pathways/security/>
- Essentials of GitHub Advanced Security
  - <https://resources.github.com/learn/pathways/security/essentials/essentials-github-advanced-security/>
- Intermediate guides to GitHub Advanced Security
  - <https://resources.github.com/learn/pathways/security/intermediate/intermediate-guides-github-advanced-security/>
- Advanced guides to GitHub Advanced Security
  - <https://resources.github.com/learn/pathways/security/advanced/advanced-module-github-advanced-security/>

## Introduction to adopting GitHub Advanced Security at scale

- <https://docs.github.com/en/enterprise-cloud@latest/code-security/adopting-github-advanced-security-at-scale/introduction-to-adopting-github-advanced-security-at-scale>

**GitHub Security Lab:** <https://securitylab.github.com/>

**GitHub CodeQL:** <https://github.com/github/codeql>



# Learning Resources

## Microsoft Learn

<https://learn.microsoft.com/en-us/collections/rqymc6yw8q5rey>

## LinkedIn Learning

<https://www.linkedin.com/learning/github-advanced-security-cert-prep-by-microsoft-press>

## GitHub Resources

<https://resources.github.com/learn/pathways/security/essentials/essentials-github-advanced-security/>

## Microsoft Learn

<https://learn.microsoft.com/en-us/collections/rqymc6yw8q5rey>

## GHAS Practice Exams

[https://ghcertified.com/practice\\_tests/advanced\\_security/?questions=116](https://ghcertified.com/practice_tests/advanced_security/?questions=116)

## Let's Book the Exam!

<https://examregistration.github.com/overview>

**Slides at** <https://github.com/PagelsR/Talks>



**Q & A**





Slides

<https://github.com/PagelsR/Talks>

# Thank you!



**Randy Pagels**

Principal Trainer

Xebia USA | Microsoft Services



Randy-Pagels

